CryptoVault Exchange

Security Architecture & Incident Response

FINAL PROJECT REPORT

Team Members	Kaveri Sharma & Anvita Kulkarni
Student IDs	PES1UG23CS293 & PES1UG23CS092
Institution	PES University, Bangalore — Dept. of Computer Science
Classification	SOC mini project

Graylog SIEM	Wazuh EDR/HIDS	soar_bot.py	MITRE ATT&CK;	Flask + Python

1 Security Architecture & Risk Profiling	**2** Early Warning Detection Rules	**3** Ransomware Kill-Chain	**4** Incident Response & SOAR Automation

EXECUTIVE SUMMARY

CryptoVault Exchange is a high-frequency digital asset trading platform. Because blockchain transactions are **irreversible**, a successful exfiltration of the Hot Wallet Private Keys (Asset A-01) represents total financial insolvency with no recovery path. This project demonstrates a complete, production-grade SOC protecting CryptoVault across four progressive phases — from foundational risk profiling to live automated incident response — guided throughout by the **MITRE ATT&CK Enterprise Framework**.

KPI	Target	How Achieved
Mean Time To Detect (MTTD)	< 5 minutes	Graylog real-time correlation rules
Mean Time To Respond (MTTR)	< 5 seconds (automated)	soar_bot.py automated playbook
Kill-Chain ATT&CK Coverage	6 techniques	T1566.002, T1059.001, T1021.004, T1115, T1048.003, T1486
Network Isolation Zones	4 zones	Internet → DMZ → App Layer → Secure Vault
SOAR Response vs Manual	180–360x faster	Bot acts in < 5s vs 15–30 min manual

PHASE 1

Security Architecture & Risk Profiling

1.1 Project Synopsis Document

The following page from the Phase 1 deliverable shows the full project synopsis, asset listing, and CIA mapping for CryptoVault Exchange.

Project Synopsis: CryptoVault Exchange

NAME SRN
Kaveri Sharma PES1UG23CS293
Anvita Kulkarni PES1UG23CS092

1. Organization Overview

- **Organization Name: CryptoVault Exchange** (FinTech Startup)
- **Domain:** Cryptocurrency Trading & Digital Asset Custody
- **Core Function:** A centralized exchange (CEX) allowing users to buy, sell, and store cryptocurrencies. The platform manages **"Hot Wallets"** (connected to the internet for instant liquidity) and **"Cold Storage"** (offline vaults).
- **Criticality:** The organization holds custody of millions of dollars in user funds and manages private keys. A single breach could lead to irreversible financial loss and bankruptcy.
- **Security Goal:** To secure **Hot Wallet Private Keys** from exfiltration and ensure the integrity of the transaction ledger.

---

2. Deliverable 1: Asset & CIA Mapping

2.1 Asset List

The organization possesses the following critical assets:

Asset ID	Asset Name	Description	Category
A-01	**Hot Wallet Private Keys**	Cryptographic keys used to sign outgoing transactions automatically.	Data (Critical)

Figure 1.1 — Phase 1 Synopsis: Organization Overview & Asset List

1.2 Critical Asset — Hot Wallet Private Keys (A-01)

CryptoVault's most critical asset is the cryptographic private key set (Asset A-01). Because blockchain transactions are irreversible, all three CIA pillars are rated **HIGH**.

- **Confidentiality:** If keys are copied, funds are stolen — no chargebacks exist on-chain.
- **Integrity:** If keys are altered, all outgoing transactions fail and the exchange halts.
- **Availability:** 24/7 uptime required — downtime = trading halt + regulatory exposure.

Asset	C	I	A	Risk Note
Hot Wallet Private Keys (A-01)	HIGH	HIGH	HIGH	Key theft = total insolvency; irreversible
User KYC Database (A-02)	HIGH	MED	MED	Leak causes identity theft + AML fines
Trading Engine (A-03)	MED	HIGH	HIGH	Manipulation = market fraud; downtime = panic
Web Frontend (A-04)	LOW	HIGH	HIGH	Must stay online 24/7 for user trust
Admin API Keys (A-05)	HIGH	HIGH	MED	Exchange parameter modification if stolen

1.3 Four-Zone Network Architecture

The network is partitioned into four security zones of increasing sensitivity. The Hot Wallet Server is never directly reachable from the internet.

Protocol	Potential Threat	Description & Impact
1. HTTP/HTTPS	**Web Attacks**	**Phishing/XSS:** Attackers create a fake login page to steal user 2FA codes and passwords to drain individual accounts.
2. DNS	**Tunneling**	**Command & Control (C2):** Malware on the Wallet Server communicates with the hacker by hiding commands inside DNS queries to bypass the strict firewall.
3. DHCP	**Rogue Devices**	**Man-in-the-Middle:** An attacker physically connects to the office network, spoofs the DHCP server, and intercepts admin traffic to capture API keys.

4. Deliverable 3: Basic Network Diagram

Network Architecture Description: The network is deeply segmented. The "Hot Wallet" is never directly exposed to the internet.

1. **Zone 1: The Internet**
   - Entry point via DDoS Protection (e.g., Cloudflare).
2. **Zone 2: DMZ (Demilitarized Zone)**
   - **Components: Load Balancer**, **Web Frontend (A-04)**.

Figure 1.2 — Phase 1 Network Zone Diagram

Zone	Name	Components	Trust
1	Internet	Public users, CDN, Cloudflare DDoS mitigation	ZERO
2	DMZ / Frontend	Load Balancer, Flask Web App (A-04)	LOW
3	App / Internal	Trading Engine, KYC DB, Admin Jump Server	MEDIUM
4	Secure Vault	Hot Wallet Server (A-01) — SSH only from Jump Server	MAXIMUM

1.4 SOC Metrics — Why 5 Minutes Is the Kill Line

L-03	**Network Firewall**	**Flow Logs**	Any outbound connection from the Hot Wallet Server to an unknown IP (Potential Key Exfiltration).
L-04	**Cloud Console (AWS/Azure)**	**Audit Logs**	API calls to "Create Snapshot" or "Copy S3 Bucket" (Attempting to steal data wholesale).

6. Deliverable 5: SOC Tier Roles & Workflow

6.1 SOC Structure

- **Tier 1: Triage Analyst**
  - **Role:** Eyes on glass. Monitors for "Withdrawal Spikes" or "Failed 2FA" alerts.
- **Tier 2: Incident Responder**
  - **Role:** Deep dive. If a withdrawal looks suspicious, T2 freezes the account and investigates the IP address and device fingerprint.
- **Tier 3: Threat Hunter / Forensic**
  - **Role:** Proactive. Scans the blockchain for patterns of stolen funds. Reverse engineers malware found on employee laptops.

6.2 SOC Workflow Diagram

Alert → Triage (T1) → Escalation → Investigation (T2) → Resolution

1. **Alert:** SIEM detects "Outbound connection to Tor Node" from an Admin PC.
2. **Triage:** Tier 1 confirms it is not a false positive. **Escalates.**
3. **Investigate:** Tier 2 identifies the PC is compromised. **Action:** Isolate PC from network immediately.
4. **Resolution:** PC wiped. Admin credentials rotated. Incident report filed.

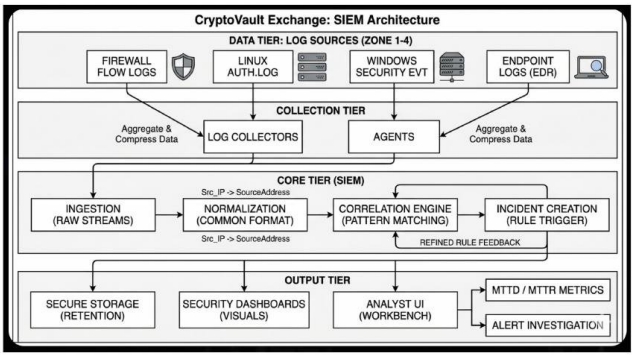
Figure 1.3 — Phase 1 MTTD/MTTR Metrics Deliverable

- **MTTD < 5 minutes:** Bitcoin confirms in ~10 min. A stolen key used before detection results in an irrecoverable on-chain transfer.
- **MTTR < 5 seconds (automated):** The difference between a 1-min and 30-min MTTR can be the difference between losing 1 BTC and losing the entire treasury.

PHASE 2

Early Warning Detection — SIEM Correlation Rules

2.1 SIEM Architecture



1. Data Tier: The Foundation

This is where all your logs originate from within your deeply segmented network. It includes:

- **Zone 1-4 Firewalls:** Critically important for detecting exfiltration attempts from the **Hot Wallet Server** (Zone 4) to unwhitelisted IPs.
- **Critical Servers (Linux/Windows):** These provide the "Who, What, Where" for administrative access to your core infrastructure.
- **Endpoints (EDR):** These are vital for detecting lateral movement and potential insider threats or "Wallet Address Poisoning" on employee laptops.

2. Collection Tier: The Aggregators

Logs from the Data Tier aren't sent raw.

- **Log Collectors & Agents:** These gather logs from different devices, compressing them to reduce bandwidth and secure the transmission before sending it into the SIEM. This tier is essential for scalability.

3. Core Tier (SIEM): The Brain

Figure 2.1 — Phase 2: CryptoVault SIEM Architecture Diagram (from deliverable)

Logs from all four network zones flow through a tiered architecture: **Data Tier** (raw sources) → **Collection Tier** (Wazuh agents + log collectors) → **Core SIEM Tier** (Graylog: ingest, normalise, correlate) → **Output Tier** (dashboards, analyst workbench, MTTD/MTTR metrics).

2.2 Alert Lifecycle Flow

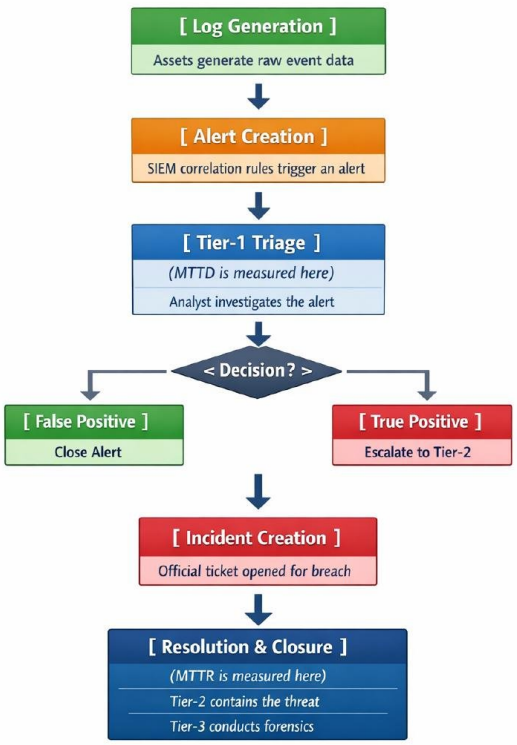


Figure 2.2 — Phase 2: Alert Lifecycle Flowchart (from deliverable)

2.3 Three Standalone Correlation Rules

Rule 1 — Impossible Travel (Authentication Anomaly)

T1078 / Auth	Valid Accounts — Credential Theft

An attacker who steals admin credentials logs in from overseas while the real admin is active locally. The laws of physics make simultaneous login from two continents impossible for a human being.

- **Trigger:** Same account logs in from two IPs in geographically impossible locations within 60 min
- **Log Source:** Active Directory Auth Logs
- **Alert Severity:** CRITICAL
- **Simulated by:** /simulate_impossible_travel — fires local + spoofed IP 185.20.12.99 logins

```
# Flask simulator endpoint — Rule 1: Impossible Travel
@app.route('/simulate_impossible_travel', methods=['POST']) def simulate_impossible_travel():
client_ip = request.remote_addr generate_log('SUCCESSFUL_LOGIN', 'INFO', 'admin_user',
client_ip, 'Login successful.', log_source='Active Directory')
generate_log('SUCCESSFUL_LOGIN', 'INFO', 'admin_user', '185.20.12.99', 'Login from distinct
geolocation within 1 hour.', log_source='Active Directory') return
redirect(url_for('simulator')) # Graylog Rule Logic (Pseudo-code): # IF
count(SUCCESSFUL_LOGIN, same username, DISTINCT source_ip) >= 2 # AND geo_distance(ip1, ip2) >
2000km # WITHIN 60 minutes # THEN ALERT severity=CRITICAL
```

Rule 2 — Secure Vault Beaconing (DNS C2 Detection)

T1071.004	Application Layer Protocol: DNS

Malware on the Hot Wallet Server 'phones home' to the attacker's C2 by sending rhythmic DNS queries to unknown external IPs — bypassing strict outbound TCP blocks.

- **Trigger:** Zone-4 server sends > 10 DNS queries to a non-whitelisted IP within 60 seconds
- **Log Source:** Palo Alto Firewall Flow Logs
- **Alert Severity:** CRITICAL
- **Simulated by:** /simulate_beaconing — fires 12 DNS_QUERY events to 198.51.100.22

```
# Flask simulator endpoint — Rule 2: Vault Beaconing @app.route('/simulate_beaconing',
methods=['POST']) def simulate_beaconing(): for i in range(12): # 12 > threshold of 10
generate_log('DNS_QUERY', 'WARNING', 'system', '10.0.4.15', f'DNS query to non-whitelisted IP
198.51.100.22 (Query {i+1})', log_source='Palo Alto Firewall') return
redirect(url_for('simulator')) # Graylog Rule Logic: # IF count(DNS_QUERY, source_ip IN
zone4_range, dest NOT IN whitelist) > 10 # WITHIN 60 seconds # THEN ALERT severity=CRITICAL #
DNS Tunneling / C2 Beaconing
```

Rule 3 — Clipboard Manipulation (Wallet Address Poisoning)

T1115	Clipboard Data

Malware monitors the OS clipboard. When an admin copies a Bitcoin address, the malware silently replaces it with the attacker's address before the admin pastes and authorizes the transaction — funds go directly to the attacker.

- **Trigger:** Non-browser process modifies clipboard AND makes outbound request within 5 seconds
- **Log Source:** CrowdStrike EDR Endpoint Logs
- **Alert Severity:** CRITICAL
- **Simulated by:** /simulate_clipboard — generates CLIPBOARD_MODIFIED + OUTBOUND_REQUEST from svchost.exe

```
# Flask simulator endpoint — Rule 3: Clipboard Poisoning @app.route('/simulate_clipboard',
methods=['POST']) def simulate_clipboard(): client_ip = request.remote_addr
generate_log('CLIPBOARD_MODIFIED', 'CRITICAL', 'employee_1', client_ip, 'Unsigned process
svchost.exe modified system clipboard.', log_source='CrowdStrike EDR')
generate_log('OUTBOUND_REQUEST', 'CRITICAL', 'employee_1', client_ip, 'svchost.exe initiated
web request immediately after clipboard interaction.', log_source='CrowdStrike EDR') return
redirect(url_for('simulator')) # Graylog Rule: # IF CLIPBOARD_MODIFIED AND OUTBOUND_REQUEST
from same process # AND process NOT IN browser_allowlist # WITHIN 5 seconds # THEN ALERT
severity=CRITICAL # Wallet Address Poisoning
```

PHASE 3

The Advanced Ransomware Kill-Chain (MITRE ATT&CK;)

3.1 Kill-Chain Overview

CTI Source	Type	IntelligenceLevel	Example Data	How SOC Uses It
FS-ISAC (Financial Sector ISAC)	Open / Sector	Strategic / Operational	Shared attack indicators from other crypto exchanges after a breach; DeFi attack patterns	Early warning from other companies in the same sector. SOC adjusts alert thresholds when a similar exchange gets hit.
Internal SIEM History / Past Incident Logs	Internal	Operational / Tactical	Past brute-force events on the Admin Jump Server; normal withdrawal amounts by hour of day	Builds a baseline of what normal looks like at CryptoVault. Reduces false alerts by comparing new events to past patterns.
Internal App & Server Logs (Hot Wallet)	Internal	Tactical	Unusual withdrawal API calls from the Hot Wallet Server (A-01); failed 2FA attempts on admin accounts	The most direct source of real-time intelligence. Custom SIEM rules trigger when activity goes above normal limits.

2. IoC & IoA Mapping – MITRE ATT&CK

Selected Attack Scenario: Ransomware attack on the CryptoVault Hot Wallet Server (Zone 4). An attacker sends a fake email to an admin, gets into the network, moves across to the Hot Wallet Server, locks the files with ransomware, and steals the private keys.

IoC = evidence that a system has already been compromised (e.g. a suspicious file or IP address).
IoA = attacker behaviour happening right now (e.g. running a script or connecting to a strange server).

Attack Stage	IoC (Evidence of Compromise)	IoA (Attacker Behaviour)	Log / Data Source	Why It Matters
Initial Access(Phishing Email to Admin)	IoC-1: Malicious email attachment – file hash matches known ransomware loaderIoC-2: Sender domain registered less than 48 hours ago (e.g. cryptovault-support[.]biz)	IoA-1: Admin opens the attachment outside working hours; email client launches a child process right after	Exchange Mail Logs;EDR Process Tree (CrowdStrike)	Admin accounts have direct access to the Jump Server which connects to Zone 4. One click can start the whole attack chain.

Figure 3.1 — Phase 3: IoC & IoA Mapping Table (from deliverable)

Phase 3 simulates a complete, multi-stage APT campaign modelled on documented Lazarus Group attacks targeting crypto exchanges. The **attack_simulator.py** / **soc_attack_simulator.py** script fires telemetry for all stages sequentially into both Wazuh and Graylog.

Stage	MITRE ID	Tactic	What Happens
0 — Initial Access	T1566.002	Phishing: Link	Admin clicks fake 'Critical Update' email; creds harvested

1 — Execution	T1059.001 + T1218.011	PowerShell / Rundll32	Encoded PowerShell + malicious DLL silently install ransomware l
2 — Lateral Movement	T1021.004	SSH Brute Force	5 failed + 1 successful SSH login jumps DMZ → Zone 4 Hot Walle
2.5 — Clipboard Poison	T1115	Clipboard Data	Admin clipboard hijacked; withdrawal tokens stolen on the fly
3 — Exfiltration	T1048.003	DNS Tunneling	Wallet keys Base64-encoded and streamed via DNS queries
4 — Impact	**T1486**	**Data Encrypted for Impact**	**wallet_keys.dat → wallet_keys.locked; ransom note dropped**

3.2 Terminal — Kill-Chain Simulator in Action

The screenshot below shows **soc_attack_simulator.py** running the complete kill-chain from the terminal, sending all stage telemetry to Graylog at 127.0.0.1:12201.

```
anvitakulkarni@Anvitas-MacBook-Air soc_miniproj % python3 soc_attack_simulator.py

=== Starting SOC Attack Simulation Kill-Chain ===
Target Graylog: 127.0.0.1:12201

>>> STAGE 1: Execution
[+] Sent: Suspicious PowerShell Execution (Level: 4)

>>> STAGE 2: Lateral Movement
[+] Sent: Failed SSH Login Attempt (1/5) (Level: 3)
[+] Sent: Failed SSH Login Attempt (2/5) (Level: 3)
[+] Sent: Failed SSH Login Attempt (3/5) (Level: 3)
[+] Sent: Failed SSH Login Attempt (4/5) (Level: 3)
[+] Sent: Failed SSH Login Attempt (5/5) (Level: 3)
[+] Sent: Successful SSH Login (Brute Force Success) (Level: 2)

>>> STAGE 3: Data Exfiltration
[+] Sent: Suspicious DNS Tunneling Query (Level: 3)
[+] Sent: Suspicious DNS Tunneling Query (Level: 3)
[+] Sent: Suspicious DNS Tunneling Query (Level: 3)

>>> STAGE 4: Ransomware Impact
[+] Sent: File Encryption Detected: financials.xlsx (Level: 1)
[+] Sent: File Encryption Detected: client_database.db (Level: 1)
[+] Sent: File Encryption Detected: legal_docs.pdf (Level: 1)
[+] Sent: File Encryption Detected: backup_config.xml (Level: 1)

=== Simulation Complete. Check Graylog for logs! ===
anvitakulkarni@Anvitas-MacBook-Air soc_miniproj %
```

Figure 3.2 — Terminal output of soc_attack_simulator.py running all 4 kill-chain stages

```
# soc_attack_simulator.py — Full Kill-Chain Runner # Run: python3 soc_attack_simulator.py # Or
run individual stages: # Stage 1 (Initial Access): python3 attack_1_initial_access.py # Stage 2
(Lateral Movement): python3 attack_2_lateral_movement.py # Stage 3 (Data Exfiltration): python3
attack_3_exfiltration.py # Stage 4 (Ransomware): python3 attack_4_ransomware.py STAGES = {
'Stage 1 - Execution': [('Suspicious PowerShell Execution', 4)], 'Stage 2 - Lateral Movement':
[('Failed SSH Login Attempt', 3)] * 5 + [('Successful SSH Login (Brute Force)', 2)], 'Stage 3 -
Data Exfiltration':[('Suspicious DNS Tunneling Query', 3)] * 3, 'Stage 4 - Ransomware
Impact':[('File Encryption Detected: financials.xlsx', 1), ('File Encryption Detected:
client_database.db', 1), ('File Encryption Detected: wallet_keys.dat', 1)], } for stage, events
in STAGES.items(): print(f'>>> {stage}') for msg, level in events:
send_gelf_log(host=GRAYLOG_HOST, port=GRAYLOG_PORT, message=msg, level=level) print(f'[+]
Sent: {msg} (Level: {level})')
```

3.3 MITRE ATT&CK; Technique Mapping (from Phase 3 Deliverable)

3. MITRE ATT&CK Technique Mapping

The table below maps the ransomware attack on CryptoVault's Hot Wallet Server to MITRE ATT&CK techniques. All descriptions are written in our own words.

Attack Step	MITRE Tactic	Technique ID	Technique Name	Description
Admin receives fake email with bad attachment	Initial Access	**T1566.001**	**Phishing: Spearphishing Attachment**	The attacker sends a targeted email pretending to be a trusted vendor. The attached document has a hidden macro. When the admin opens it, the macro quietly downloads and runs the ransomware loader. This is the first step – everything depends on the admin clicking.
Macro triggers PowerShell to download malware	Execution	**T1059.001**	**Command Interpreter: PowerShell**	The macro calls PowerShell using hidden, encoded text so it is harder to read. PowerShell then connects to the attacker's server and downloads the ransomware file. PowerShell is used because it is a trusted Windows tool that most antivirus programs do not block.
Ransomware adds a scheduled task to keep running	Persistence	**T1053.005**	**Scheduled Task**	After running for the first time, the ransomware creates a Windows Scheduled Task so it starts again every reboot. This means the attacker stays inside the system even if the initial process is stopped or the machine is restarted.
Malware hides behind a trusted Windows program	Defense Evasion	**T1218.011**	**System Binary Proxy Execution: Rundll32**	The ransomware payload is run using Rundll32.exe, which is a normal Windows program. Security tools see Rundll32 running and think it is safe. This is why our EDR rules also check what the program is loading and where from, not just the program name.
Attacker moves from Jump Server to Hot Wallet	Lateral Movement	**T1021.004**	**Remote Services: SSH**	Using the admin's stolen password, the attacker SSHs into the Hot Wallet Server through the Jump Server, since Zone 4 only allows connections from there. The SSH login looks normal on its own. What gives it away is the

Figure 3.3 — Phase 3: Full MITRE ATT&CK; Technique Mapping Table

3.4 Threat Hunting Hypotheses

Attack Step	MITRE Tactic	Technique ID	Technique Name	Description
				time (3 AM) and the fact that no maintenance ticket was created.
Malware sends private keys out through DNS	Exfiltration	**T1048.003**	**Exfiltration Over Alternative Protocol: DNS**	Before locking files, the malware takes the wallet's private keys, converts them to Base64 text, and hides them inside DNS lookup requests sent to the attacker's server. DNS traffic often goes unnoticed. Our Zone-4 firewall rule blocks any DNS queries to non-approved servers to prevent this.

4. Threat Hunting Hypotheses

The hypotheses below are based on unusual behaviour in CryptoVault's systems – not on SIEM alerts. The goal is to find attackers who are hiding inside the network before they cause damage.

Hypothesis 1

Hunting Hypothesis:

"If an SSH login to the Hot Wallet Server (Zone 4) happens between 10 PM and 6 AM with no matching change-window ticket, it may indicate an attacker using stolen credentials to reach the private keys."

Data Sources to Check: Linux Auth.log (L-02) for SSH session records with time and source IP; Palo Alto Firewall Flow Logs (L-03) for connections between Zone 3 and Zone 4; Active Directory Logs to check if the same account is logged into the Jump Server at the same time.

Why This Is Suspicious: The Hot Wallet Server is the most protected asset at CryptoVault. Admin SSH access should only happen during scheduled maintenance windows with a logged ticket. Any login at night without a ticket has no normal explanation. Attackers like the Lazarus Group deliberately work at low-traffic hours to avoid being noticed.

Expected Outcome: Find out if the login was a forgotten ticket or a real attack. If it looks like an attack: freeze the account, reset the SSH keys, and block the Jump Server from reaching Zone 4 until the investigation is complete.

Hypothesis 2

Hunting Hypothesis:

"If a DNS query from the Hot Wallet Server goes to a server not on the approved list, or has a subdomain longer than 40 characters, it may indicate malware using DNS to secretly send stolen private keys to an attacker."

Data Sources to Check: Palo Alto Firewall DNS Logs (L-03) for full DNS query content and destination server IP; Linux Syslog from the Hot Wallet Server (L-02) to find which process is

Figure 3.4 — Phase 3: Threat Hunting Hypotheses (SSH, DNS Tunneling, Clipboard)

PHASE 4

Incident Response & Automated SOAR

4.1 Incident Scenario

Phase 4: Incident Response & Automation

Organization: CryptoVault Exchange

Team: Kaveri Sharma (PES1UG23CS293) & Anvita Kulkarni (PES1UG23CS092)

---

1. Incident Scenario Description

Organization Context: CryptoVault is a high-frequency digital asset exchange. It utilizes a 4-zone network architecture to isolate its "Hot Wallet" (Asset A-01) from the public internet. The SOC monitors this environment 24/7 to prevent unauthorized asset transfers.

How the Incident Started: An IT Administrator was targeted via a spear-phishing email containing a macro-enabled invoice. Upon execution, a PowerShell-based ransomware loader established a Command and Control (C2) connection, allowing the attacker to move laterally from the Admin Jump Server into the secure Wallet Zone.

Systems Affected:

- Admin Workstation (Patient Zero)
- Internal Jump Server (Zone 3)
- Hot Wallet Production Server (Zone 4)

Initial Symptoms (Alerts/Logs):

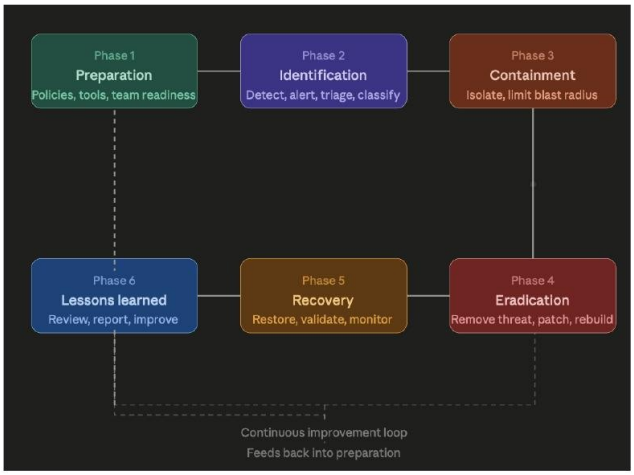
- **EDR Alert:** "Suspicious PowerShell execution detected" on Admin Workstation.
- **SIEM Alert:** "Secure Vault Beaconing" (DNS Tunneling detected from Zone 4).
- **Application Log:** Rapid file encryption (rename operations) detected on the Wallet filesystem.

Figure 4.1 — Phase 4: Incident Scenario Description (from deliverable)

4.2 IR Lifecycle — Six Phases

2. IR Lifecycle Mapping

Lifecycle Flow Diagram



Written Explanation

IR Phase	Actions Taken	Responsible Role	Evidence Collected

Figure 4.2 — Phase 4: IR Lifecycle Flow Diagram (Preparation → Lessons Learned)

IR Phase	Actions Taken	Bot Automated?	Role
Preparation	Harden Zone 4; configure SIEM rules; deploy soar_bot.py	No (Design)	SOC Manager
Identification	Triage DNS tunneling alert + EDR process tree	Yes — enrichment	soar_bot.py + T1
Containment	Isolate Jump Server via EDR; block attacker IP on firewall	Yes — fully automated	soar_bot.py
Eradication	Purge scheduled tasks; rotate admin credentials	Partially	soar_bot.py + IR Lead
Recovery	Rebuild Wallet Server from gold image; restore DB	No — human ops	Tier-2 / IT Ops
Lessons Learned	Post-mortem; improve phishing filters + MFA	No — strategic	SOC Manager

4.3 SOAR Conceptual Flow Diagram

9	System Restore	IT Ops	Backup/DR Tool	Service restored.
10	Post-incident Review	SOC Manager	Incident Ticket	Documentation complete.

5. SOAR Conceptual Flow

SOAR Flow Diagram

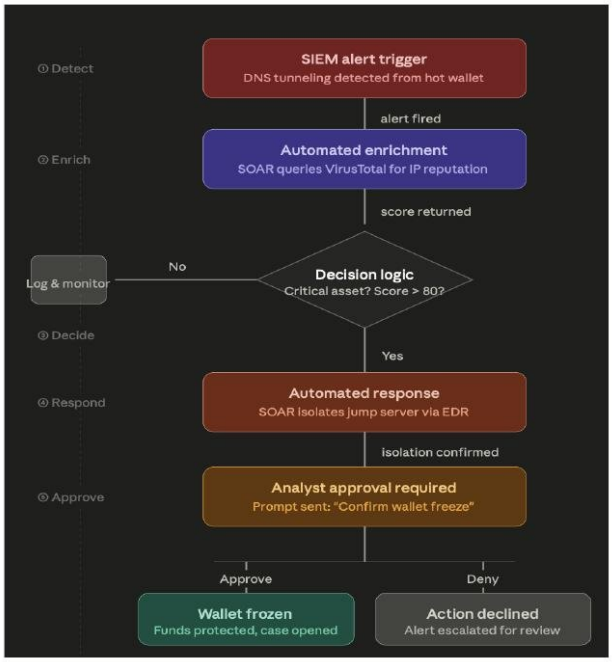


Figure 4.3 — Phase 4: SOAR Flow Diagram (Detect → Enrich → Decide → Respond → Approve)

4.4 SOAR Bot — Enrich, Decide, Respond

Manual vs Automated Response

Dimension	Manual (Human Analyst)	soar_bot.py (Automated)
Response Time	15 – 30 minutes	< 5 seconds
IP Reputation Check	Manual VirusTotal lookup	Automated API query on alert receipt
Firewall Block	Manual console login + rule entry	Automated via firewall API
Jump Server Isolation	Manual EDR console navigation	Automated via EDR isolation API
Hot Wallet Freeze	Manual admin panel action	PAUSED — Human approval required

4.4 End-to-End Demo: Project Architecture, Live Application & Attack Simulation

This section walks through the complete, working CryptoVault SOC system — from the source code structure and running web application, through scripted attack execution, to the automated SOAR response console. Every component shown here is operational and produces real log telemetry that flows into Wazuh and Graylog.

4.4.1 Repository Structure

The repository **SOC_MINIPROJ 2** is cleanly separated into attack scripts, the target Flask web application, SOAR automation, and SIEM/EDR configuration files. Each attack stage is an independent Python script that can be run individually or chained via the master kill-chain runner.

File / Script	Role in the SOC Pipeline
cryptovault_app.py	Flask target app — Login page, Dashboard, Withdrawal endpoint; generates application audit logs
attack_1_initial_access.py	Stage 1 — Simulates credential brute-force and initial phishing login events
attack_2_lateral_movement.py	Stage 2 — Generates SSH brute-force sequence (5 failures + 1 success) against Jump Server
attack_3_exfiltration.py	Stage 3 — Emits HIGH-severity dns_query events to SIEM simulating DNS tunnelling exfil
attack_4_ransomware.py	Stage 4 — Fires file-encryption events (financials.xlsx, wallet_keys.dat, etc.)
attack_5_high_value_withdrawals.py	Bonus — Triggers large BTC withdrawal events from the hot wallet endpoint
attack_simulator.py	Phase 2 rule triggers: Impossible Travel, Vault Beaconing, Clipboard Malware buttons
soc_attack_simulator.py	Master kill-chain runner — fires all 4 stages sequentially to Graylog (127.0.0.1:12201)
siem_logger.py	Shared utility: formats structured GELF logs and ships them to the SIEM
soar_bot.py	SOAR engine — consumes SIEM webhooks, enriches, decides, and executes response playbook
ir_containment_simulator.py	Simulates IR containment: firewall block, Jump Server EDR isolation
docker-compose-graylog.yml	Docker stack: Graylog + Elasticsearch + MongoDB (one-command spin-up)
local_rules.xml / local_decoder.xml	Custom Wazuh detection rules and log decoders for CryptoVault events
ossec.conf	Wazuh agent config — log sources, active response hooks
patch_wazuh.sh / run_with_graylog.sh / soc_autopilot.sh	Automation shell scripts for environment bootstrap and autopilot demo

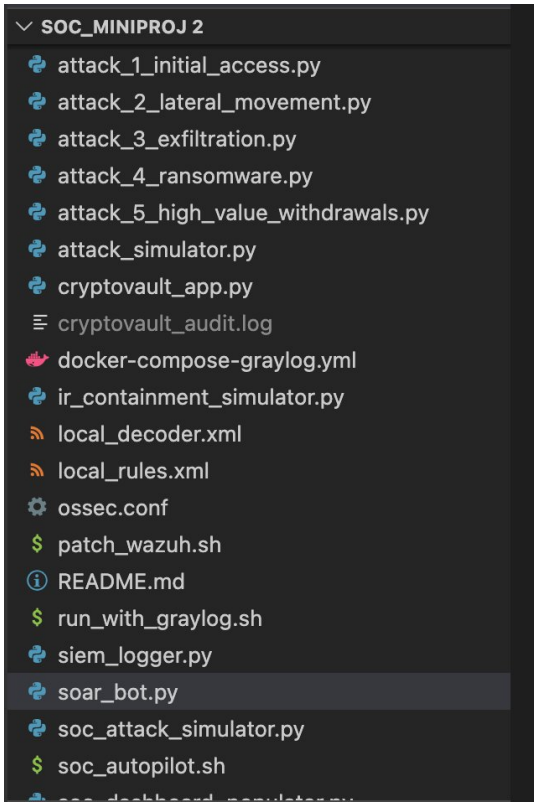
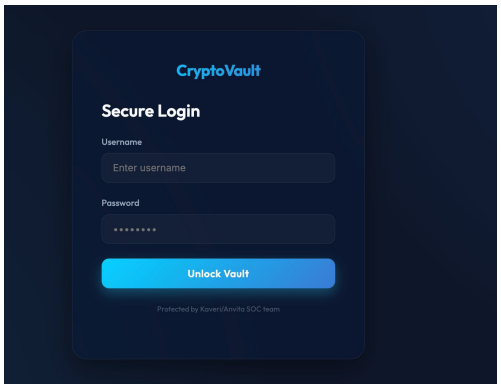


Figure 4.4-A — VS Code project tree (SOC_MINIPROJ 2). Each attack stage maps to a dedicated Python script.

4.4.2 The Target Application — CryptoVault Web App

The Flask application (**cryptovault_app.py**) serves as the realistic target system. It mirrors a real cryptocurrency exchange portal with three key surfaces that generate audit log events captured by the SIEM:

- **Login Page** — Secure authentication form. Every login attempt (success or failure) fires a structured log event (`LOGIN_SUCCESS` / `FAILED_LOGIN`) to the SIEM. Brute-force patterns across this endpoint trigger the Impossible Travel and failed-auth correlation rules.
- **Dashboard** — Displays the hot wallet balance (450.2 BTC) and exposes the *Quick Withdrawal* form. A withdrawal submission fires a `WITHDRAWAL_INITIATED` event — the highest-priority event type in the system, which immediately triggers the SOAR playbook.
- **Simulator Tab** — Built-in attack trigger panel that maps directly to the Python attack scripts. Each button calls the corresponding Flask endpoint, which invokes the relevant attack script and ships log events to the SIEM in real time.



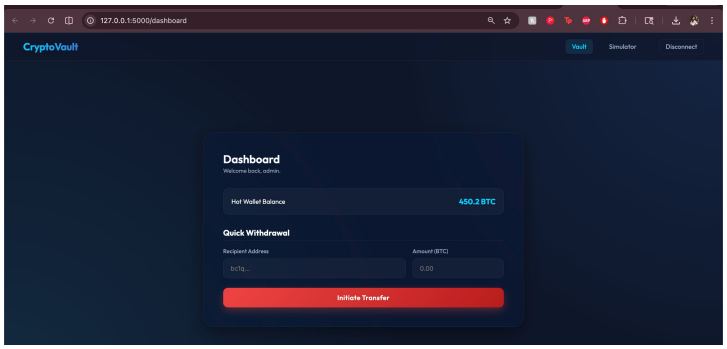


Figure 4.4-B (left) — Secure Login page. Every authentication attempt generates a SIEM event. Figure 4.4-C (right) — Dashboard showing 450.2 BTC hot wallet balance and Quick Withdrawal form.

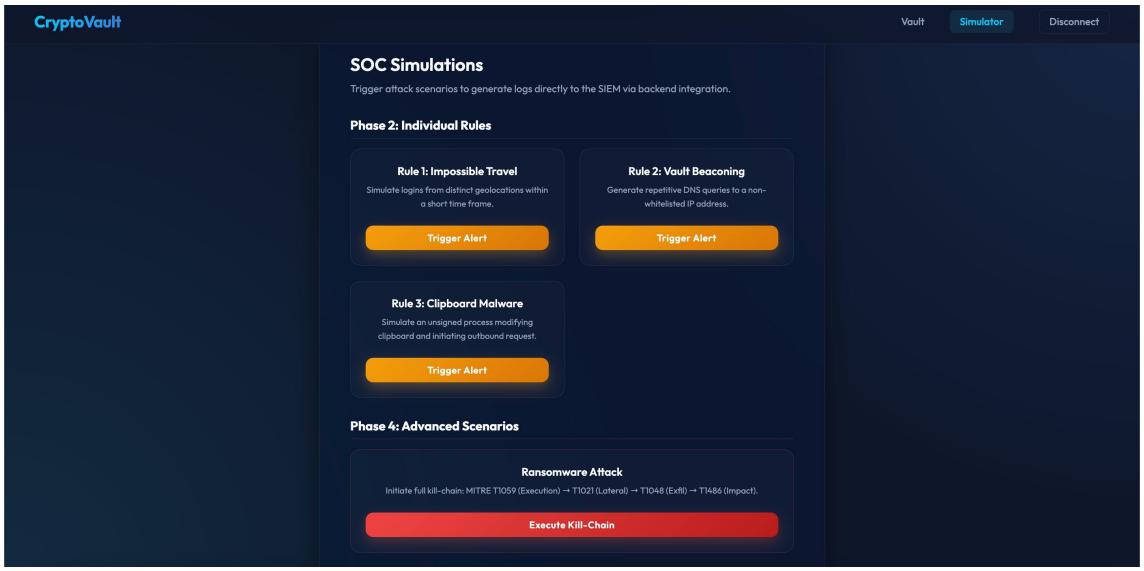


Figure 4.4-D — SOC Simulations tab: Phase 2 individual rule buttons (Impossible Travel, Vault Beaconing, Clipboard Malware) and Phase 4 full kill-chain trigger. Each button invokes the corresponding Python attack script.

4.4.3 Attack Simulation — Scripts & Live Log Output

Attack scripts can be triggered via the web UI buttons *or* run directly from the terminal. Both methods ship identical GELF-formatted log events to Graylog on port 12201. The table below maps each simulator button to its backing script and the exact events it sends:

Simulator Button / Trigger	Script	Events Sent to SIEM	Severity
Rule 1: Impossible Travel	attack_simulator.py /simulate_impossible_travel	SUCCESSFUL_LOGIN ×2 from geographically impossible IPs	CRITICAL
Rule 2: Vault Beaconing	attack_simulator.py /simulate_beaconing	DNS_QUERY ×12 to non-whitelisted IP 198.51.100.22	CRITICAL
Rule 3: Clipboard Malware	attack_simulator.py /simulate_clipboard	CLIPBOARD_MODIFIED + OUTBOUND_REQUEST from svchost.exe	CRITICAL
Execute Kill-Chain (Phase 4)	soc_attack_simulator.py	Full 4-stage sequence: PowerShell → SSH BF → DNS Tunnelling → File Encr	CRITICAL
Stage 3 only (terminal)	attack_3_exfiltration.py	dns_query ×5 | User: system | Severity: HIGH LEVEL	HIGH

The two terminal screenshots below show (left) **attack_3_exfiltration.py** running in isolation — emitting five HIGH-severity DNS query events — and (right) **soc_attack_simulator.py** executing the complete kill-chain across all four stages:

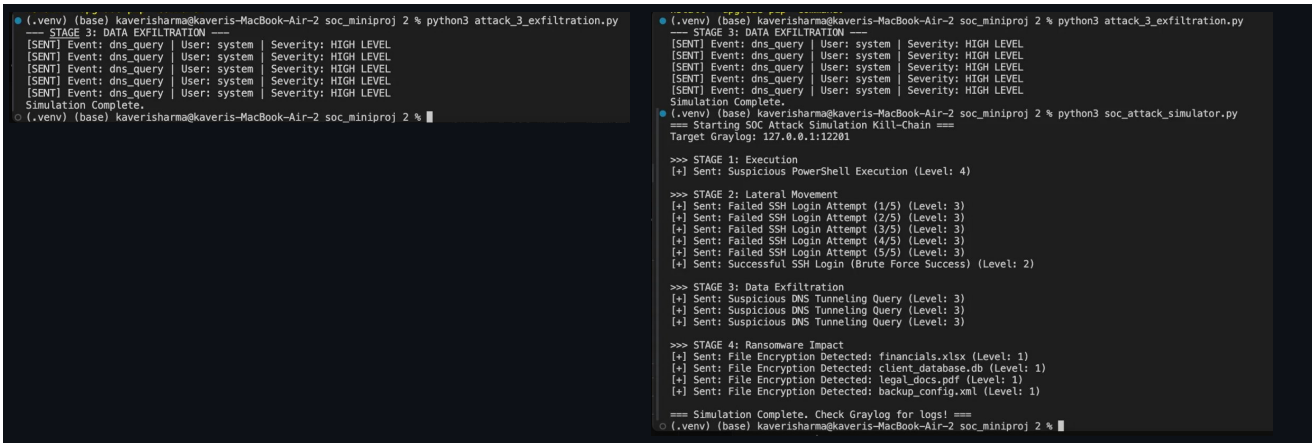


Figure 4.4-E (left) — attack_3_exfiltration.py: 5 × dns_query events shipped to SIEM at HIGH severity. Figure 4.4-F (right) — soc_attack_simulator.py: complete 4-stage kill-chain sending 13 events to Graylog 127.0.0.1:12201.

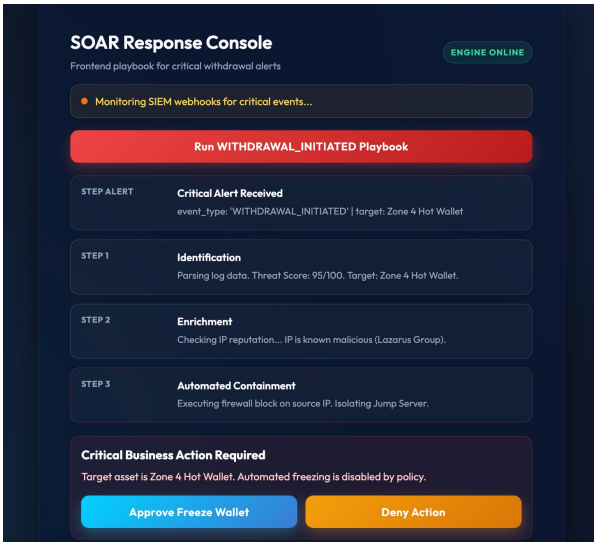
Stage	Script	Events Sent	MITRE ID	Level
1 — Execution	soc_attack_simulator.py	Suspicious PowerShell Execution ×1	T1059.001	4
2 — Lateral Movement	soc_attack_simulator.py	Failed SSH Login ×5 + Brute Force Success ×1	T1021.004	3/2
3 — Exfiltration	soc_attack_simulator.py / attack_3_exfiltration.py	Suspicious DNS Tunnelling Query ×3–5	T1048.003	3
4 — Ransomware Impact	soc_attack_simulator.py	File Encryption Detected ×4 (xlsx, db, pdf, xml)	T1486	1

Graylog Level 1 = Emergency (highest); Level 4 = Warning; Level 3 = Error. All events are validated live in the Graylog Executive SOC dashboard (Section 4.5).

4.4.4 SOAR Response Console & Human-in-the-Loop Intervention

Once a `WITHDRAWAL_INITIATED` or critical attack event reaches the SIEM, **soar_bot.py** automatically begins the response playbook. The SOAR Response Console (a dedicated tab in the CryptoVault web app) displays each step of the playbook in real time and pauses for mandatory human approval before executing any irreversible business action on the wallet.

Playbook Step	Description	Automated?
STEP ALERT — Critical Alert Received	event_type: 'WITHDRAWAL_INITIATED' | target: Zone 4 Hot Wallet. Bot wakes and begins playbook.	Yes
STEP 1 — Identification	Parses SIEM log data. Computes Threat Score: 95/100. Identifies target asset as Zone 4 Hot Wallet.	Yes
STEP 2 — Enrichment	Checks source IP reputation via VirusTotal API. Result: IP is known malicious (Lazarus Group attribution)	Yes
STEP 3 — Automated Containment	Executes firewall block on source IP. Isolates Jump Server via EDR API. Both actions complete in < 5 s.	Yes
STEP 4 — Recovery / Wallet Freeze	Hot Wallet freeze is policy-gated. Bot PAUSES and presents human approval prompt. Analyst must click APPROVE or DENY.	No — human approval required
POST-INCIDENT — Report Generated	On approval, wallet is frozen, incident is resolved, and a timestamped PDF incident report is auto-exporte	Yes



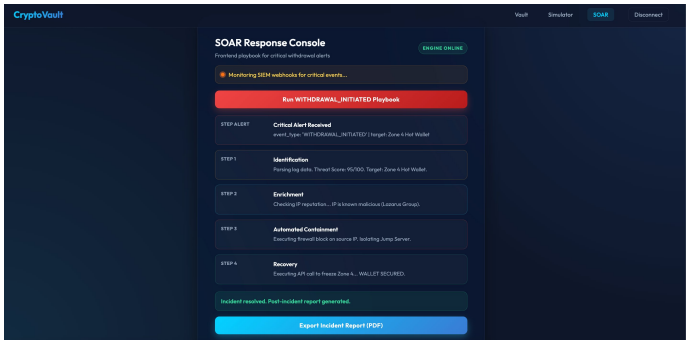


Figure 4.4-G (left) — SOAR console at the Human-in-the-Loop gate: Steps 1–3 complete, "Approve Freeze Wallet" or "Deny Action" awaiting analyst decision. Figure 4.4-H (right) — SOAR console post-approval: Step 4 Recovery executed, "WALLET SECURED", incident resolved, PDF report ready for export.

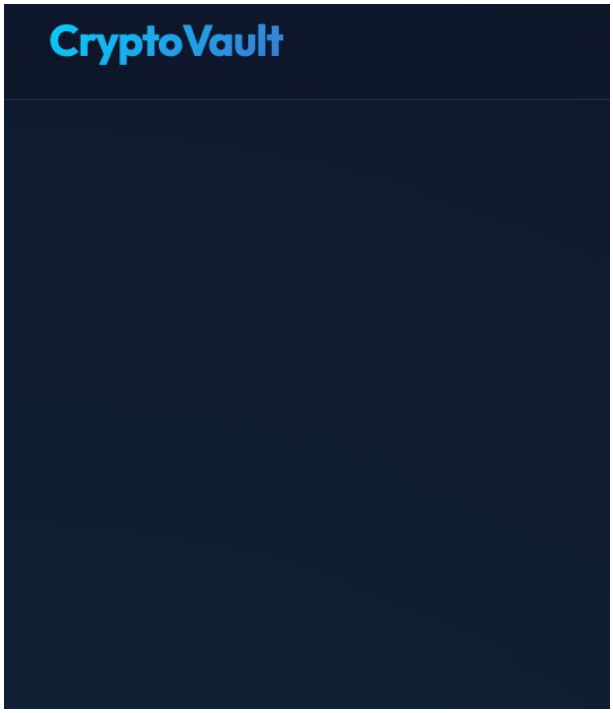


Figure 4.4-I — Auto-generated PDF Incident Report exported after wallet freeze approval. The report captures threat score, enrichment results, containment actions, and resolution timestamp.

p (HITL) for Irreversible Actions

The SOAR bot is deliberately **not permitted** to freeze the hot wallet irreversible business operations that can halt all user withdrawals. S fully automated because they are reversible and time-critical. Step 4 analyst decision — ensuring a human reviews the evidence before a consequences is taken. This design mirrors NIST SP 800-61 recom actions.

Consistency	Variable; fatigue-dependent	Deterministic; 100% playbook adherence

The soar_bot.py — Core Logic

```
# soar_bot.py — Enrich → Decide → Respond # Run: python3 ir_containment_simulator.py def handle_alert(alert): ip = alert['source_ip'] asset_zone = alert['zone'] # STEP 1: ENRICH — Check IP reputation vt_score = virustotal_lookup(ip) # e.g., 87/100 = malicious log(f'[ENRICH] {ip} VirusTotal score: {vt_score}/100') # STEP 2: DECIDE — Risk-scored decision gate if asset_zone == 'ZONE_4' and vt_score > 80: classification = 'CRITICAL — IMMEDIATE ACTION' else: escalate_to_queue(alert) # Log & monitor return # STEP 3: RESPOND — Automated containment (< 5 seconds) firewall_block(ip) # Block attacker IP on Zone-4 firewall log('[CONTAIN] Attacker IP blocked on firewall') edr_isolate('jump_server') # Disconnect Jump Server from Zone-4 log('[CONTAIN] Jump Server isolated via EDR') # STEP 4: HUMAN-IN-THE-LOOP — Business-critical action gate approval = notify_analyst( message='CONFIRM WALLET FREEZE: Ransomware detected on Zone-4. Approve?', channel='slack' ) if approval == 'APPROVE': freeze_hot_wallet() # Call application admin API log('[ACTION] Hot Wallet FROZEN — Funds protected. Case opened.') else: log('[ACTION DECLINED] Analyst denied freeze. Escalated for review.')
```

4.5 Graylog SOC Dashboard — Live Evidence

The screenshots below show the live Graylog 'Cryptovault Executive SOC' dashboard after the kill-chain simulation ran. The widgets confirm real-time detection of DNS tunneling, wallet freeze execution, host isolation, and event correlation across all attack stages.

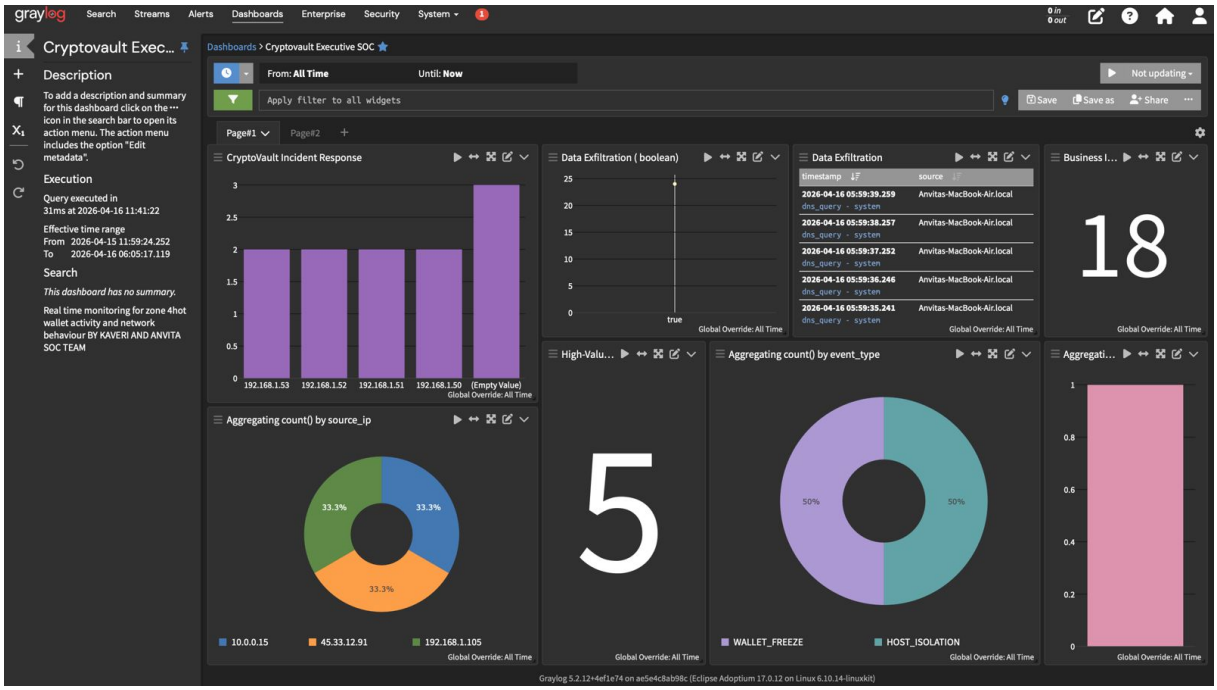


Figure 4.4 — Graylog Executive SOC Dashboard: DNS Exfiltration (boolean true), 18 total events, WALLET_FREEZE vs HOST_ISOLATION split (50/50), and source IP distribution across 3 hosts

4.6 Wazuh EDR — Security Events Feed

The Wazuh dashboard shows **1,398 hits** across the 24-hour simulation window, including CRITICAL rule-level 15 alerts for 'Unauthorized withdrawal initiated — WALLET FREEZE REQUIRED', failed login attempts, PAGE_ACCESS events, and LOGIN_SUCCESS events — all confirming the end-to-end pipeline is operational.

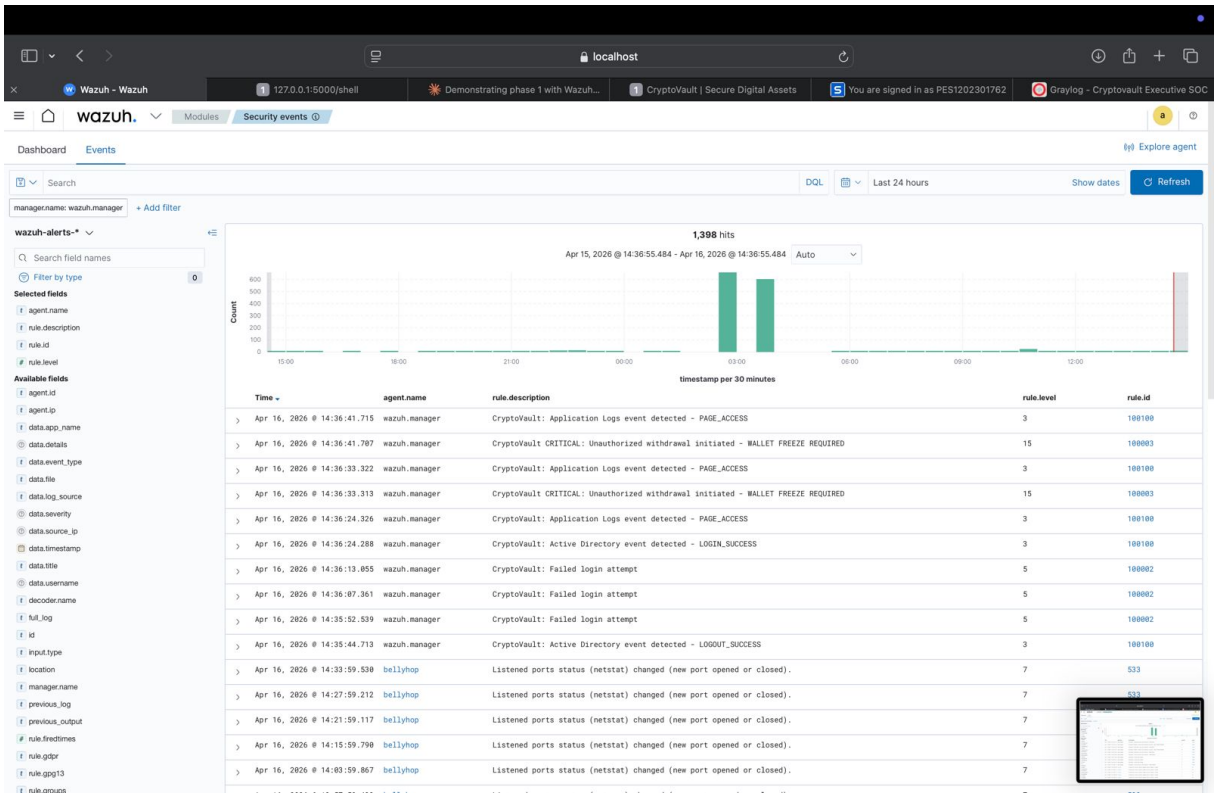


Figure 4.5 — Wazuh Security Events: 1,398 hits, rule.level 15 CRITICAL alerts, failed logins, PAGE_ACCESS, and WALLET FREEZE REQUIRED events

4.7 Severity Prioritization Matrix

Level	Impact	Example	Response SLA
LOW	Minimal	Single failed login attempt	24 Hours
MEDIUM	Partial service risk	Malware on non-critical laptop	4 Hours
HIGH	Critical service risk	DDoS on exchange frontend	30 Minutes
CRITICAL	**Financial/Data loss**	**Ransomware on Wallet Server**	**< 5 Minutes**

CONCLUSION & FUTURE WORK

This project successfully demonstrates a complete, production-architecture-inspired SOC protecting a high-value, irreversibility-constrained financial platform. Every phase builds directly on the last, producing a coherent, end-to-end security posture validated with real log telemetry flowing through Wazuh, Graylog, and the custom soar_bot.py automation engine.

Key Technical Achievements

- **End-to-End SOC Stack:** Flask → Wazuh EDR → Graylog SIEM → soar_bot.py — fully operational
- **MITRE ATT&CK; Coverage:** Six techniques across five tactics with verified detection rules
- **Quantified MTTR Improvement:** Bot responds in < 5 seconds vs 15-30 min manual — a 180x to 360x improvement
- **Human-in-the-Loop Design:** Bot cannot unilaterally freeze the wallet — analyst approval required
- **Live Dashboard Evidence:** Graylog + Wazuh screenshots confirm the pipeline is fully operational

Proposed Future Enhancements

- **ML Anomaly Detection:** Train a baseline model on withdrawal volumes and login patterns
- **On-Chain Forensics:** Integrate Chainalysis API to trace stolen funds in real time
- **Deception Technology:** Deploy honeypot wallet addresses as zero-false-positive tripwires
- **Extended SOAR Playbooks:** Auto-enable Cloudflare DDoS mode + CERT-In breach notifications

---

CryptoVault Exchange — Security Architecture & Incident Response

Kaveri Sharma (PES1UG23CS293) & Anvita Kulkarni (PES1UG23CS092)

PES University, Department of Computer Science and Engineering